COLLEGE OF COMPUTING, INFORMATICS AND MATHEMATICS

OPTIMIZATION ALGORITHMS AND APPLICATION (ISP611)

CLASS : CDCS2595A

INDIVIDUAL ASSIGNMENT

TITTLE:

OPTIMIZING OF SPORTS TOURNAMENT SCHEDULING USING GENETIC ALGORITHMS

PREPARED BY :

NURUL FADZLIN BINTI MUSTAFA KAMAR

(2023448868)

PREPARED FOR : PROFESOR MADYA DR MARINA YUSOFF

DATE OF SUBMISSION : 13 DECEMBER 2024

Table of Contents

1.0 Case Study..... 1

2.0 Objective Function.....2

3.0 Chromosome Representation/ Solution Representation.....3

4.0 Programming Language.....5

5.0 Genetic Algorithm Steps in Coding Representation.....6

6.0 Results and Discussion.....11

7.0 Suggestion For Improvement.....16

8.0 Conclusion..... 17

9.0 References.....18

1.0 Case Study

Scheduling in sports may be quite difficult , especially when there are several leagues with common infrastructure, A notable research addresses practical limitations such venue capacity, shared resources, and league-to-league fairness in context of multi-league sports scheduling. In order to demonstrate the effects and remedies for these issues, this case study uses actual situations, particularly from the Belgian national football association.

Teams from different leagues frequently play in the same clubs and at the same locations in amateur and youth sports. because of things like available pitches and scheduling slots, each venue has a restricted capacity. Because clubs must prevent venue capacity from being exceeded, scheduling games concurrently across hundreds of leagues creates interdependencies. When leagues have diverse team sizes and need different amounts of rounds, this problem is even worse (Li et al.,2023).

In order to overcome these obstacles, using the genetic algorithms approach, the researchers created a mixed-integer programming (MIP) model and heuristic techniques to produce workable schedules. In an effort to reduce capacity violations, the model assigns home-away patterns (HAPs) to each team as efficiently as possible. The approach respects venue restrictions while guaranteeing fairness in the allocation of home and away games by employing round-robin configurations (Davari et al., 2020). The heuristic approach incorporates local search and perturbation strategies to refine initial schedules. This method proved efficient in computational experiments, providing high-quality solutions for both artificial and real-world datasets. For example, schedules generated for the Belgian football leagues demonstrated significant reductions in capacity violations without necessitating substantial infrastructure investments (Li et al., 2023)

The findings of these studies emphasize the importance of integrated scheduling for multi-league sports. League organizers can guarantee equality, equity and effectively manage resources and reduce disruptions brought on by capacity problems by implementing sophisticated optimization techniques. Additionally, these strategies' performance in Belgium implies other situations with comparable logistic and structural limitations (LI et al.,2023).

2.0 Objective Function

The objective function is the mathematical expression across the reviewed papers is to minimize capacity violations and/or travel distances for multi-league sports scheduling, often while maintaining fairness in scheduling. General Formula:

$$\text{Minimize } v_{IP} = \sum_{c \in C} \sum_{r \in R} z_{c,r} + \alpha \sum_{t \in T} \sum_{p \in O_t} \sum_{r \in R} d(t,p) \cdot y_{t,p,r}$$

Where;

- $\boldsymbol{v_{IP}}$ is the Total objective value.
- $\boldsymbol{Z_{c,r}}$ is the capacity violation for club c in round r.
- $\boldsymbol{d(t,p)}$ is the distance between teams t and p.
- $\boldsymbol{Y_{t,p,r}}$ is the binary variable indicating whether the team t plays against p in round r.
- $\boldsymbol{C}$ is the set of all clubs.
- $\boldsymbol{R}$ is the set of rounds.
- $\boldsymbol{T}$ is the set of teams.
- $\boldsymbol{O_t}$ is the eligible opponents for team t.
- $\boldsymbol{\alpha}$ is the eight factor to balance the two objectives.

Explanation of the Objective function:

- Minimizing Capacity Violations. The term $\sum c \in C \sum r \in R\ Z_{c,r}$ ensures that the number of matches scheduled at each club in a round does not exceed its capacity. $Z_{c,r}$ captures excess matches beyond capacity and is minimized.
- Minimizing The Travel Distance. The term $\sum t \in T \sum p \in O_t \sum r \in R\ d(t,p) \cdot Y_{t,p,r}$ ensures that the total distance traveled by all teams is reduced. $Y_{t,p,r}$ ensures matches are assigned between eligible opponents.

- Balancing Constraints. The constraints promote fairness by ensuring that each team has a well-balanced schedule, incorporating both home and away games, while also complying with league-level and venue-level requirements.

3.0 Chromosome Representation/ Solution Representation

A potential solution to the scheduling problem is usually represented as a chromosome in genetic scheduling algorithms. Many times, each chromosome is encoded as a series of genes, each of which is linked to a certain match or tournament event.

1. **Assignment of teams to Home-Away Patterns (HAPs)**: Determines when a team plays home or away.
2. **Match schedules**: Specifies which teams play against each other in each round.

For example, if a tournament with:

- $T = \{t_1, t_2, t_3, t_4\}$
- $R = \{r_1, r_2\}$
- $HAPs = \{h_1, h_2\}$

A chromosome could be encoded as:

Chromosome: [(t_1,h_1), (t_2,h_2), (t_3,h_1), (t_4,h_2); (t_1,t_2,r_1), (t_3,t_4,r_1)]

The first part is assigned to the HAPs to Teams, and the second part assigned for matches to rounds.

Attributes:
Each gene in the chromosome corresponds to a match scheduled, time slots and rounds.
Example Representation:

Team	HAP Assignment	Round 1 (Time Slot 90s)	Round 2 (Time Slot 180s)	Round 3 (Time Slot 270s)	Round 4 (Time Slot 360s)	Round 5 (Time Slot 450s)
Team 1	HAP 1 ("H", "A", "H", "A", "H")	vs Team 2 (H) @ 90s	vs Team 3 (A) @ 180s	vs Team 4 (H) @ 270s	vs Team 5 (A) @ 360s	vs Team 6 (H) @ 450s
Team 2	HAP 2 ("A", "H", "A", "H", "A")	vs Team 1 (A) @ 90s	vs Team 4 (H) @ 180s	vs Team 5 (A) @ 270s	vs Team 6 (H) @ 360s	vs Team 3 (A) @ 450s
Team 3	HAP 1 ("H", "A", "H", "A", "H")	vs Team 4 (H) @ 90s	vs Team 5 (A) @ 180s	vs Team 6 (H) @ 270s	vs Team 1 (A) @ 360s	vs Team 2 (H) @ 450s
Team 4	HAP 2 ("A", "H", "A", "H", "A")	vs Team 3 (A) @ 90s	vs Team 6 (H) @ 180s	vs Team 1 (A) @ 270s	vs Team 2 (H) @ 360s	vs Team 5 (A) @ 450s
Team 5	HAP 1 ("H", "A", "H", "A", "H")	vs Team 6 (H) @ 90s	vs Team 1 (A) @ 180s	vs Team 2 (H) @ 270s	vs Team 3 (A) @ 360s	vs Team 4 (H) @ 450s
Team 6	HAP 2 ("A", "H", "A", "H", "A")	vs Team 5 (A) @ 90s	vs Team 2 (H) @ 180s	vs Team 3 (A) @ 270s	vs Team 4 (H) @ 360s	vs Team 1 (A) @ 450s

Explanation of Chromosome Representation

1. Encoding matches and patterns which The HAP or match assignment of a team is represented by each gene on the chromosome. The task guarantees adherence to fairness and venue capacity restrictions.
2. Fitness evaluation,The ability of chromosomes to reduce the objective function capacity violations and trip distances is the basis for evaluation.

4.0 Programming Language

Python is a great choice for this project because of its robust libraries, ease of use, and flexibility. Implementation is rapid and easy thanks to its user-friendly syntax, while sophisticated computations and data management are made simpler by tools like Pandas and NumPy. Clear visuals are provided by libraries like Matplotlib, which aid in the efficient interpretation of results. Because of these characteristics, Python is perfect for optimization jobs like tournament scheduling.

Furthermore, Python is quite flexible, enabling the addition of additional features and limitations as the project develops. Access to resources and updates is made possible by the robust community support, and its open-source nature guarantees cost effectiveness. If necessary, Python's compatibility with sophisticated technologies like TensorFlow and Scikit-learn allows it to incorporate predictive elements. The combination of Python's scalability and ease of use makes it a dependable and future-ready platform.

5.0 Genetic Algorithm Steps in Coding Representation

No	Step in GA	Coding

1	**Calculate Distance Matrix** - Generates the matrix with distances between teams. Each entry in the matrix corresponds to the travel distance between team	`import numpy as np`
		`import random`
		`import matplotlib.pyplot as plt`
		`import pandas as pd`
		
		`# Read the dataset`
		`df =`
		`pd.read_csv('Sample_Game_1_RawEventsData-Copy1.csv')`
		
		`# Parameters`
		`num_teams = 6`
		`num_rounds = 5`
		`venue_capacity = 3`
		`population_size = 20`
		`generations = 50`
		`mutation_rate = 0.1`
		
		`def calculate_distance_matrix(num_teams):`
		`  """Step 1: Calculate Distance Matrix"""`
		`  distance_matrix = np.random.randint(1, 50, size=(num_teams, num_teams))`
		`  np.fill_diagonal(distance_matrix, 0)`
		`  return distance_matrix`

2	**Initialization** - Initializes a population of solutions (chromosomes). Each chromosome represents a potential schedule	`def initialize_population():`
		`  """Step 2: Initialization"""`
		`  population = []`
		`  for _ in range(population_size):`
		`  chromosome = {`
		`  "hap_assignments": np.random.choice(len(HAPs), num_teams).tolist(),`
		`  "schedule": [`
		`  random.sample(range(num_teams), 2)`

		``
		for _ in range(num_rounds)
		            ],
		        }
		        population.append(chromosome)
		    return population
		``
		
		``
		``
		``

	chromosomes based on fitness.Selection methods such as tournament selection or roulette wheel selection ensure that fitter solutions have a higher chance of reproducing, while still maintaining diversity by giving weaker solutions a small chance to survive.	`population.sort(key=lambda x: evaluate_fitness(x, distance_matrix))` `return population[:population_size // 2]`

5	**Crossover -** Combines two parents to create offspring.A crossover point is randomly chosen, and parts of each parent's HAP assignments and match schedules are exchanged to produce new chromosomes.	`def crossover(parent1, parent2):`
		`"""Step 5: Crossover"""`
		`crossover_point = random.randint(1, num_teams - 1)`
		`child1 = {`
		`"hap_assignments": parent1["hap_assignments"][:crossover_point] + parent2["hap_assignments"][crossover_point:],`
		`"schedule": parent1["schedule"][:crossover_point] + parent2["schedule"][crossover_point:],`
		`}`
		`child2 = {`
		`"hap_assignments": parent2["hap_assignments"][:crossover_point] + parent1["hap_assignments"][crossover_point:],`
		`"schedule": parent2["schedule"][:crossover_point] + parent1["schedule"][crossover_point:],`
		`}`
		`return child1, child2`

6	**Mutation -** Introduces random changes to offspring	`def mutate(chromosome):`
		`"""Step 6: Mutation"""`
		`if random.random() < mutation_rate:`

	to maintain diversity.This step makes random changes to parts of a chromosome	`team_idx = random.randint(0, num_teams - 1)`
		`chromosome["hap_assignments"][team_idx] = random.randint(0, len(HAPs) - 1)`
		`if random.random() < mutation_rate:`
		`round_idx = random.randint(0, num_rounds - 1)`
		`chromosome["schedule"][round_idx] = random.sample(range(num_teams), 2)`
		`return chromosome`

7	**Replacement** - Replaces the old population with the new offspring.This step ensures the algorithm progresses by focusing on the newly generated solutions while discarding older, less fit solutions.	`def replace_population(population, new_population):`
		`"""Step 7: Replacement"""`
		`return new_population`

8

		`while len(new_population) < population_size:`
		`parent1, parent2 = random.sample(selected_population, 2)`
		`child1, child2 = crossover(parent1, parent2)`
		`new_population.append(mutate(child1))`
		`new_population.append(mutate(child2))`
		
		`# Replace the population with new offspring`
		`population = replace_population(population, new_population)`
		
		`# Track the best solution`
		`for chromosome in population:`
		`fitness = evaluate_fitness(chromosome, distance_matrix)`
		`if fitness < best_fitness:`
		`best_fitness = fitness`
		`best_solution = chromosome`
		
		`fitness_history.append(best_fitness)`
		`print(f"Generation {generation}: Best Fitness = {best_fitness}")`
		
		`return best_solution, fitness_history`

10	**Show Results and Plot** - Displays the results and plots the fitness history over generations.A plot of fitness values over generations provides a visual representation of the optimization process, showing how the algorithm improved	`def plot_results(fitness_history):`
		`"""Step 10: Show Result and Plot"""`
		`plt.plot(fitness_history)`
		`plt.title("Fitness Over Generations")`
		`plt.xlabel("Generation")`
		`plt.ylabel("Fitness")`
		`plt.show()`
		
		`# Home-Away Patterns (HAPs)`
		`HAPs = [`

	the solutions over time.	```
		    ["H", "A", "H", "A", "H"],
		    ["A", "H", "A", "H", "A"],
		]
		
		# Run the Genetic Algorithm
		best_solution, fitness_history =
		genetic_algorithm()
		plot_results(fitness_history)
		print("Best Solution:", best_solution)
		``

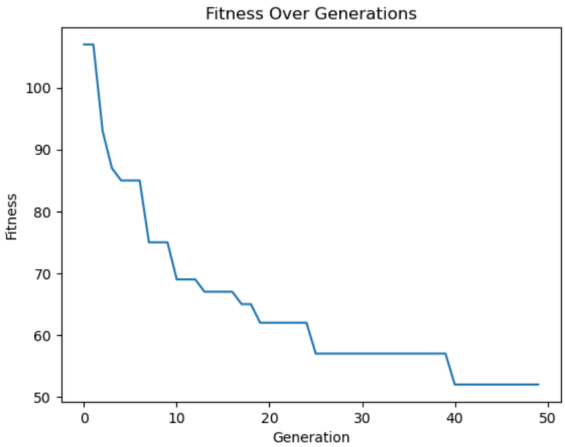
6.0 Results and Discussion

No. Test	Change(Parameter)	Result(Output)
1	num_teams = 6 num_rounds = 5 venue_capacity = 3 population_size = 20 generations = 50 mutation_rate = 0.1	Generation 0: Best Fitness = 107 Generation 1: Best Fitness = 107 Generation 2: Best Fitness = 93 Generation 3: Best Fitness = 87 Generation 4: Best Fitness = 85 Generation 5: Best Fitness = 85 Generation 6: Best Fitness = 85 Generation 7: Best Fitness = 75 Generation 8: Best Fitness = 75 Generation 9: Best Fitness = 75 Generation 10: Best Fitness = 69 Generation 11: Best Fitness = 69 Generation 12: Best Fitness = 69 Generation 13: Best Fitness = 67 Generation 14: Best Fitness = 67 Generation 15: Best Fitness = 67

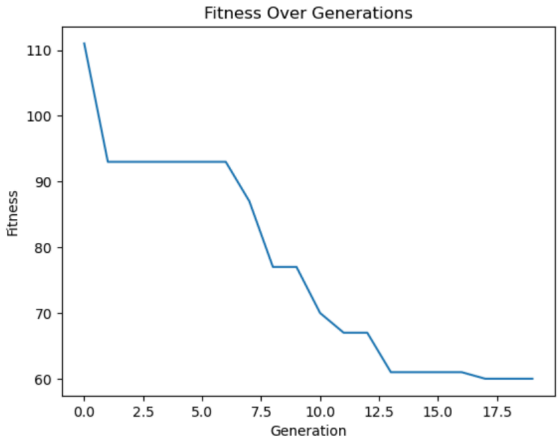
		Generation 16: Best Fitness = 67 Generation 17: Best Fitness = 65 Generation 18: Best Fitness = 65 Generation 19: Best Fitness = 62 Generation 20: Best Fitness = 62 Generation 21: Best Fitness = 62 Generation 22: Best Fitness = 62 Generation 23: Best Fitness = 62 Generation 24: Best Fitness = 62 Generation 25: Best Fitness = 57 Generation 26: Best Fitness = 57 Generation 27: Best Fitness = 57 Generation 28: Best Fitness = 57 Generation 29: Best Fitness = 57 Generation 30: Best Fitness = 57 Generation 31: Best Fitness = 57 Generation 32: Best Fitness = 57 Generation 33: Best Fitness = 57 Generation 34: Best Fitness = 57 Generation 35: Best Fitness = 57 Generation 36: Best Fitness = 57 Generation 37: Best Fitness = 57 Generation 38: Best Fitness = 57 Generation 39: Best Fitness = 57 Generation 40: Best Fitness = 52 Generation 41: Best Fitness = 52 Generation 42: Best Fitness = 52 Generation 43: Best Fitness = 52 Generation 44: Best Fitness = 52 Generation 45: Best Fitness = 52

		Generation 46: Best Fitness = 52 Generation 47: Best Fitness = 52 Generation 48: Best Fitness = 52 Generation 49: Best Fitness = 52 Best Solution: {'hap_assignments': [1, 1, 0, 1, 0, 0], 'schedule': [[0, 3], [0, 3], [1, 3], [5, 4], [5, 4]]}
2	num_teams = 6 num_rounds = 5 venue_capacity = 3 population_size = 10 generations = 20 mutation_rate = 0.5	Generation 0: Best Fitness = 111 Generation 1: Best Fitness = 93 Generation 2: Best Fitness = 93 Generation 3: Best Fitness = 93 Generation 4: Best Fitness = 93 Generation 5: Best Fitness = 93 Generation 6: Best Fitness = 93 Generation 7: Best Fitness = 87 Generation 8: Best Fitness = 77 Generation 9: Best Fitness = 77 Generation 10: Best Fitness = 70 Generation 11: Best Fitness = 67 Generation 12: Best Fitness = 67 Generation 13: Best Fitness = 61 Generation 14: Best Fitness = 61 Generation 15: Best Fitness = 61 Generation 16: Best Fitness = 61 Generation 17: Best Fitness = 60 Generation 18: Best Fitness = 60 Generation 19: Best Fitness = 60 Best Solution: {'hap_assignments': [1, 1, 1, 0, 0, 0], 'schedule': [[3, 4], [3, 4], [1, 2], [3, 4], [0, 1]]}

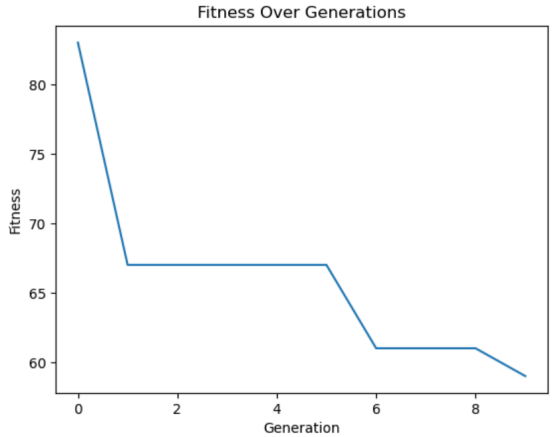
3	num_teams = 6 num_rounds = 5 venue_capacity = 3 population_size = 30 generations = 10 mutation_rate = 0.15	Generation 0: Best Fitness = 83 Generation 1: Best Fitness = 67 Generation 2: Best Fitness = 67 Generation 3: Best Fitness = 67 Generation 4: Best Fitness = 67 Generation 5: Best Fitness = 67 Generation 6: Best Fitness = 61 Generation 7: Best Fitness = 61 Generation 8: Best Fitness = 61 Generation 9: Best Fitness = 59 Best Solution: {'hap_assignments': [0, 1, 0, 1, 0, 1], 'schedule': [[3, 5], [0, 2], [3, 5], [0, 4], [1, 5]]}
4	num_teams = 6 num_rounds = 5 venue_capacity = 3 population_size = 25 generations = 5 mutation_rate = 0.2	Generation 0: Best Fitness = 77 Generation 1: Best Fitness = 77 Generation 2: Best Fitness = 62 Generation 3: Best Fitness = 57 Generation 4: Best Fitness = 57 Best Solution: {'hap_assignments': [1, 1, 0, 1, 0, 0], 'schedule': [[4, 1], [4, 2], [5, 4], [4, 3], [5, 4]]}



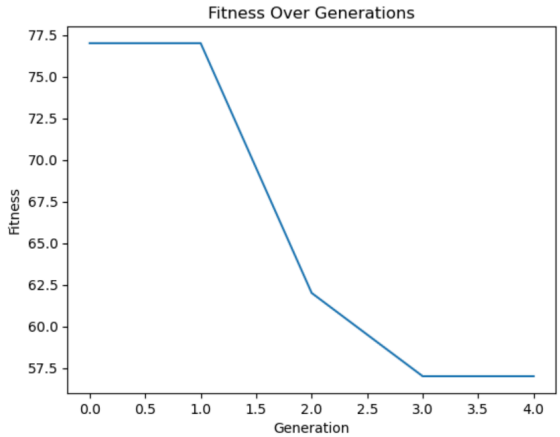
Test 1



Test 2



Test 3



Test 4

The combined plots show how well the Genetic Algorithm (GA) optimizes the sports scheduling issue across a range of test situations. The algorithm's iterative improvement of solutions is evident in the obvious trend of fitness values improving over generations in all tests.

The output graphs show how well the Genetic Algorithm optimizes sports scheduling. As solution quality improves over generations, fitness scores for all tests decrease. The computer can swiftly find better schedules through selection, crossover, and mutation, as seen by the sharp declines in fitness seen in early generations. The pace of improvement slows down as generations go by, finally stabilizing when the algorithm approaches near-optimal solutions.

Different generation limitations have an influence, as seen by the difference in results among tests. Faster convergence is the focus of shorter tests like Test 4, whereas longer exams like Test 1 allow for more complex answers. Overall, the result shows that the Genetic Algorithm can balance optimization and efficiency to provide better schedules within the limits.

7.0 Suggestion For Improvement

Although the Genetic Algorithm (GA) currently used for scheduling multi-league sports makes notable advancements, it still has room for growth. First, using hybrid optimization techniques, such as GA with integer Programming or Simulated Annealing, may accelerate convergence and result in schedules of greater quality. In order to solve certain elements of the scheduling issues, such as fairness, or lowering travel lengths, these techniques can take advantage of the advantages of algorithms.

Second, the optimizer would become more useful and flexible if it included real-world data, such as current traffic or weather conditions. One way to guarantee seamless operations during the tournament is to modify schedules in reaction to real-time data.

Third, the technique may be greatly accelerated by using parallel processing, particularly for bigger competitions with many teams and restrictions. The optimizer can investigate a larger solution space faster by measuring fitness and carrying out genetics concurrently.

Lastly, improving the visualizations tools can yield more insightful information about the scheduling procedure. Users would be better able to comprehend the algorithm's output and make necessary modifications if comprehensive graphs or interactive dashboards were included to track progress and examine timetables.

These recommendations point to ways to improve the GA-based optimizer even further, making it quicker, more flexible, and more appropriate for scheduling problems encountered in the real world.

8.0 Conclusion

In conclusion, the use of genetic algorithms (GA) to scheduling multi-league sports is a major breakthrough in resolving one of the trickiest and most computationally intensive optimization issues. This project demonstrates how to use evolutionary operations like selection, crossover and mutation, as well as dynamic chromosome representations and multi-objective fitness evaluation, to address important issues like venue capacity, home-away pattern fairness and travel distance minimization.

The program systematically improves schedules across generations thanks to the methodical GA procedure. The optimizer exhibits its versatility and scalability for practical applications by integrating intricate restrictions such as multi-league relationships and exact time windows. Understanding the algorithm's efficiency and convergence is further aided by the display of fitness gains over generations.

this GA-based strategy is superior to manual or heuristic scheduling techniques in tern of automation, quality and scalability, which makes it perfect for managing major competitions. Future developments might increase its functionality and speed even more by including real-time data, parallel computing or hybrid optimization strategies.

Overall this study demonstrates the strengths and adaptability of genetic algorithms in handling challenging optimization issues in addition to offering useful and effective solutions for sports scheduling. For future study and use in scheduling, logistics and other domains needing comparable multi-objective optimization, it establishes a strong basis.

9.0 References

Davari, M., Goossens, D., Beliën, J., Lambers, R., & Spieksma, F. C. R. (2020). The multi-league sports scheduling problem, or how to schedule thousands of matches. *Operations Research Letters*, *48*(2), 180–187. https://doi.org/10.1016/j.orl.2020.02.004

Kendall, G., Knust, S., Ribeiro, C. C., & Urrutia, S. (2010). Scheduling in sports: An annotated bibliography. In *Computers and Operations Research* (Vol. 37, Issue 1, pp. 1–19). https://doi.org/10.1016/j.cor.2009.05.013

Li, M., Davari, M., & Goossens, D. (2023). Multi-league sports scheduling with different leagues sizes. *European Journal of Operational Research*, *307*(1), 313–327. https://doi.org/10.1016/j.ejor.2022.10.010

Li, M., Van Bulck, D., & Goossens, D. (2024). Beyond leagues: A single incomplete round robin tournament for multi-league sports timetabling. *European Journal of Operational Research*. https://doi.org/10.1016/j.ejor.2024.11.007

Dataset:

metrica-sports. (2020). *sample-data/data/Sample_Game_1/Sample_Game_1_RawEventsData.csv at master · metrica-sports/sample-data*. GitHub. https://github.com/metrica-sports/sample-data/blob/master/data/Sample_Game_1/Sample_Game_1_RawEventsData.csv